Branching Strategy for Enterprise Teams

Git Flow vs. Trunk-Based Development — when to use each and how to migrate

- This talk covers the **two dominant models** for feature-driven application teams
- After this you will know which model fits your team and what migration looks like
- Audience: teams delivering features, not OSS library maintainers

Which Branch is Always Releasable?

The most important question to ask about any branching model:

Dimension	Git Flow	Trunk-Based Development
Stable branch	`main` — production-tagged only	`main` (trunk) — always releasable
Integration branch	`develop` — unreleased, accumulated work	None — trunk IS the integration surface
Deployment frequency	Limited by release stabilization cycles	On demand — trunk is always deployable
DORA profile	Common in medium/low performers	2.3x more likely in elite performers (DORA 2024)

> **Key difference:** In Git Flow, `main` is stable *after* a release ceremony.
> In TBD, trunk is stable *always* — guaranteed by CI, not by process.

Git Flow: Five Branch Types

Git Flow defines five branch types with explicit, non-negotiable roles:

```
main              ← production-only; every commit is a release tag
develop           ← integration branch; "next release" in progress
feature/ABC-123   ← individual feature work; branches off develop
release/2.3.0     ← release stabilization; branches off develop
hotfix/fix-null   ← emergency fix on production; branches off main
```

- `main` and `develop` are **permanent** — they exist for the life of the repository
- `feature/*`, `release/*`, and `hotfix/*` are **temporary** — deleted after merge
- `main` = released code only; every commit has a version tag
- `develop` = next release accumulation; integrated but may contain unreleased bugs

Git Flow: Four Merge Choreographies

Every release and hotfix requires merges in multiple directions:

Operation	Direction	Description
Feature merge	`feature/*` → `develop`	Completed feature integrates into next release
Release merge	`release/*` → `main` + tag	Stabilized release ships to production
Hotfix merge	`hotfix/*` → `main` + tag	Emergency fix ships from production branch
Back-merge	`main` or `release/*` → `develop`	Stabilization fixes flow back to integration branch

Every release requires merging in **both directions** (`develop → release → main` AND `main → develop` back-merge). Miss the back-merge and branches diverge permanently — a bug fixed in `v2.3.0` silently reappears in `v2.4.0`.

Git Flow: Where It Breaks Down

Three structural failure modes that appear at scale:

- **Long-lived feature branches** — two branches touching the same files for three weeks have zero awareness of each other's changes; conflicts compound and are resolved under deadline pressure by people without the original context
- **Merge debt** — `develop` drifts from `main`; back-merges after release become risky events rather than routine operations; teams informally freeze `develop` during release stabilization, creating dead time across the whole team
- **CI/CD friction** — every deployment requires a branch, a merge ceremony, a tag, and a reverse merge; at weekly deploys this is manageable; at daily deploys it is a bottleneck; at continuous deployment it is impossible (Thoughtworks: "Hold" on Technology Radar)

Trunk-Based Development: One Direction

TBD's defining rule: **every developer integrates to trunk (main) at least once per day.**

```
main (trunk)        ← always releasable
feature/short       ← max 1–2 days; merge before it diverges
release/2.3.0       ← cut from trunk days before ship; cherry-pick fixes
```

- No `develop` branch — trunk IS the integration surface
- Short-lived branches prevent merge debt; the branch is a **code review artifact**, not a development isolation unit
- Release branches are optional and lightweight — cut just-in-time, not weeks early
- Fixes always go to trunk first, then cherry-pick to release branch — never fix on release and merge back

Feature Flags: How TBD Ships Incomplete Features

Code is merged to trunk before a feature is complete — flags gate *exposure*, not *existence*.

Type	Purpose	Lifespan
Release flag	Hide incomplete work; code is in production but unreachable	Days–weeks; clean up after full rollout
Experiment / A/B flag	Route user cohorts to different paths; measure outcomes	Days–weeks; until statistical significance
Ops flag (kill switch)	Disable non-critical functionality during incidents	Permanent standing circuit breaker
Permission flag	Gate features for premium tier, beta users, enterprise customers	Permanent (business rule, not scaffolding)

(Fowler's canonical taxonomy — four distinct types with different owners and lifecycles)

Flag Debt: The Hidden Cost

Flag lifecycle: **Create → Dark Launch → Canary → Full Release → Cleanup**

Stage	What Happens
Dark launch	Code in production, flag off for all users; verify no latency/error regressions
Canary rollout	1% → 5% → 25% → 50% → 100%; monitor error rates at each step; rollback = flag toggle
Full release	Flag on for 100%; feature shipped; flag now has **zero business value**
Cleanup	Delete the conditional, remove the old code path, delete the flag from the store

- Release flags **must** be cleaned up post-GA — they accumulate as technical debt
- Stale flags = dead code paths that still require test coverage and still risk runtime errors
- Knight Capital Group lost $460 million in 45 minutes in 2012 — root cause: a decommissioned toggle not removed from the codebase

Two Features in Flight: Git Flow vs. TBD

Git Flow — isolation by branch, integration at the end:

```
# Week 1: Both branch off develop with no knowledge of each other
git checkout -b feature/user-auth       # Developer A — 3 weeks
git checkout -b feature/payment-redesign  # Developer B — 3 weeks
# Both touch src/app/router.ts
# Sprint end: merge conflicts resolved by someone without the original context
```

TBD — isolation by flag, integration daily:

```
# Both developers merge to trunk every 1-2 days
# feature/user-auth behind flags.isEnabled('user-auth')
# feature/payment-redesign behind flags.isEnabled('payment-redesign')
# No conflict accumulation — each PR is small and current
```

The conflict does not disappear in TBD — it is surfaced immediately, while the two developers still have context to resolve it in minutes rather than hours.

Migrating from Git Flow to Trunk: Four Stages

Stage	Focus	What Changes
Phase 0: Fix CI (weeks 1–4)	Infrastructure	CI on every PR; < 2% flaky; `main` branch-protected; do not change branching yet
Phase 1: Shorten branches (weeks 4–8)	Habits	Feature branches must merge within 5 days; break features into smaller tickets; observe friction
Phase 2: Introduce flags (weeks 6–12)	Tooling	Pick 2-3 features; implement behind flags; team learns flag lifecycle; PM learns to control rollout via flags
Phase 3: GitHub Flow (weeks 8–16)	Model	Branch from `main` not `develop`; eliminate `develop`; let it drain naturally
Phase 4: Enforce TBD discipline (months 4–6)	Polish	1-2 day branch lifetime enforced; all hotfixes to trunk first; cherry-pick to release branches

Key prerequisite before Phase 2: CI must be fast (< 10 min) and reliable (< 2% flaky).

Which Model Fits Your Team?

Criterion	Git Flow fits when...	TBD fits when...
Deployment frequency	Monthly/quarterly release cadence	Weekly or more frequent deploys desired
CI maturity	No reliable automated CI yet	CI is fast (< 20 min), reliable (< 2% flaky), enforced
Feature flag tooling	No flag system in place	Flag infrastructure available at any level
Release cadence	Formal QA sign-off phase; regulated environment	Continuous deployment; SaaS product
Team scale	Distinct QA org; multiple supported major versions	Single product; feature teams with ownership

TBD is not "better" — it is a better fit for teams that deploy frequently and have CI investment. Git Flow is a better fit for regulated environments, packaged software, and teams without automated test infrastructure.

What You Now Know

Concept	Git Flow	TBD
Stable branch	`main` — only after release merge ceremony	`main` (trunk) — always, enforced by CI
Parallel features	Long-lived branches, isolated until done	Short-lived branches + feature flags
Release process	`develop → release branch → main + tag → back-merge`	Cut from trunk days before ship; cherry-pick fixes
CI requirement	Useful but optional	Non-negotiable prerequisite
Feature isolation	Branch	Flag

- **Migration** is a phased process: fix CI first, then shorten branches, then introduce flags, then eliminate `develop`
- **Flags** are the enabling mechanism — without them, TBD forces all features to be complete before merging
- **Do not skip prerequisites** — attempting TBD without fast reliable CI produces trunk instability and reversion

📄 Reference doc: `docs/enterprise/release-strategy.md`
📊 Diagrams: `docs/enterprise/diagrams.md`